Optimisation bi-objectif traînée / SER

Front de Pareto exact, NSGA-II et boucle QUBO :
implémentation, résultats et analyse de la chaîne complète du hackathon quantique

Notes de travail, projet *HackathonQuantique*

Version du 4 novembre 2026

Avant-propos

Objet. Ce cours ferme la boucle du hackathon. Les cours précédents ont construit le modèle aérodynamique (phase 1), un modèle SER simplifié (phase 2) et les outils quantiques (QUBO, recuit, QAOA). Ici, on assemble tout : on pose le problème bi-objectif, on calcule son front de Pareto exact par énumération, on le résout par NSGA-II (l'optimisation classique de référence de la note d'intention), puis par la boucle itérative « modèle quadratique → QUBO → solveur », et on compare les méthodes en nombre de simulations, ce qui est la question posée par le hackathon.

Ce qu'on suppose connu. Le contenu des trois autres cours, en particulier les chapitres QUBO, recuit et QAOA du cours quantique. Les notions d'optimisation multi-objectif et d'algorithmes évolutionnaires sont posées depuis le début.

Fil conducteur. Le problème (ch. 1) → dominance et front de Pareto (ch. 2) → le front exact par énumération (ch. 3) → NSGA-II : principe et implémentation (ch. 4), puis résultats (ch. 5) → la boucle QUBO bi-objectif (ch. 6) → la comparaison de toute la chaîne (ch. 7) → limites et pistes (ch. 8).

Figures et chiffres. Toutes les courbes sont calculées par le code du projet (`Code/optim/`) via `tools/make_figures.py`, à partir des tables exhaustives et du fichier `resultats/pipeline.json` produit par `python3 -m optim.experiences_pipeline`. Chaque chapitre se termine par des exercices (corrigés en annexe A) et des **pistes d'amélioration**.

Itérations du projet. Le projet avance par itérations, chacune avec sa couleur dans tous les cours et dans la fiche projet (dossier `Fiche_Projet/` du projet). Chaque chapitre commence par un bandeau qui indique l'itération de son contenu ; un ajout ou une révision faite lors d'une itération ultérieure apparaît dans un encadré « Évolution » de la couleur correspondante.

V0	Modèle de la note d'intention	24–30 oct. 2026
V1	Chaîne complète conforme à la note	4–5 nov. 2026
V2	Problème reformulé	5–6 nov. 2026
V3	Concurrent équitable, problème plus grand	à partir du 6 nov. 2026

Table des matières

Avant-propos **i**

1 Le problème d'optimisation **1**

1.1 Formulation . . . 1

1.2 Deux scénarios . . . 1

1.3 Une simulation, un compteur . . . 1

Exercices . . . 2

2 Dominance et front de Pareto **3**

2.1 Dominance . . . 3

2.2 Ramener le problème à un seul objectif : scalarisation . . . 3

2.3 Mesurer la qualité d'un front approché . . . 4

Exercices . . . 4

3 Le front exact par énumération **6**

3.1 Pourquoi on peut, et pourquoi c'est précieux . . . 6

3.2 Les deux fronts . . . 6

3.3 Lecture physique . . . 7

Exercices . . . 8

4 NSGA-II : principe et implémentation **9**

4.1 Pourquoi un algorithme génétique . . . 9

4.2 Le principe . . . 9

4.3 Les deux critères de classement . . . 10

4.4 Les opérateurs, sur des variables entières . . . 10

4.5 Vérification contre pymoo . . . 11

Exercices . . . 11

5 NSGA-II sur notre problème **13**

5.1 Le protocole . . . 13

5.2 Le réglage qui compte : la taille de population . . . 13

5.3 Convergence . . . 14

Exercices . . . 14

6 La boucle QUBO bi-objectif **15**

6.1 L'algorithme . . . 15
6.2 Étape 1 : les modèles de substitution . . . 15
6.3 Étape 2 : un QUBO par poids . . . 16
6.4 Étape 3 : résoudre et choisir les candidats . . . 16
6.5 Étape 4 : simuler, archiver, recommencer . . . 17
Exercices . . . 17

7 Comparaison de toute la chaîne **19**

7.1 Le protocole . . . 19
7.2 Les courbes de convergence . . . 19
7.3 Le tableau de synthèse . . . 19
7.4 Lecture . . . 21
7.5 Ce que coûterait une vraie machine . . . 21
Exercices . . . 22

8 Limites et pistes **23**

8.1 Ce que ces résultats ne disent pas . . . 23
8.2 Passage à l'échelle . . . 23
8.3 Pistes, par rapport utilité / effort . . . 24

9 V2 : le problème reformulé **25**

9.1 Ce qui change . . . 25
9.2 Le front exact . . . 25
9.3 La comparaison des méthodes . . . 26
9.4 Lecture . . . 27
Exercices . . . 28

A Corrigés des exercices **29**

Chapitre 1

Le problème d'optimisation

V1 Itération V1 — Chaîne complète conforme à la note 4–5 nov. 2026

1.1 Formulation

La note d'intention pose « un problème bi-objectif sous contraintes de portance et d'épaisseur minimales ». Sur la grille discrète partagée par toutes les méthodes, il s'écrit :

$$\min_{(i_m,i_p,i_t)\in\{0,\dots,7\}^3} \Big(C_D(m,p,t),\ \mathrm{SER}(m,p,t,\alpha)\Big) \quad \text{sous} \quad \begin{cases} t \geq t_{\min} = 8\ \%, \\ C_L \geq C_{L,\min} = 0{,}3 \quad (\text{à } \alpha = 2^\circ \text{ imposé}), \\ \text{modèle aérodynamique valide.} \end{cases}$$

— Les **variables** sont trois indices entiers de 0 à 7 : $m \in \{0,\dots,7\}$ %, $p \in \{0{,}20,\dots,0{,}55\}$, $t \in \{6,\dots,20\}$ %. Cela fait 512 profils, codés sur 9 bits.
— Le **premier objectif** est la traînée de profil (en counts) du modèle panneaux + couche limite, à $Re = 3 \cdot 10^6$ et $\alpha = 2^\circ$.
— Le **second objectif** est la SER moyenne (en dBm) sur un secteur d'élévations, calculée par le modèle d'optique physique à 10 GHz, à l'attitude donnée par l'aérodynamique.
— Les **contraintes** viennent de la note d'intention (portance, épaisseur) et du domaine de validité du modèle aérodynamique.

1.2 Deux scénarios

	S1 : approche de face	S2 : radar en contrebas
secteur d'élévation	$[-20^\circ, 20^\circ]$	$[-60^\circ, 0^\circ]$
origine	note d'intention	cas d'étude supplémentaire
rôle	référence du hackathon	front plus riche, pour éprouver les méthodes

S2 a été ajouté après avoir constaté que S1 donne un front très court en SER (chapitre 3). Les deux scénarios partagent tout le reste : grille, point de vol, contraintes, modèles.

1.3 Une simulation, un compteur

Une « simulation » est l'évaluation d'un profil par les deux modèles, aérodynamique puis SER. C'est l'unité de coût du hackathon : la note d'intention compare les méthodes en *nombre de simulations*

nécessaires pour atteindre le front exact. Toutes les méthodes passent donc par un même objet, `CountedProblem`, qui ne compte chaque profil distinct qu'une fois.

Dans le code : `optim/problem.py`.

```python
class CountedProblem:
    def __call__(self, idx):
        idx = tuple(int(i) for i in idx)
        if idx not in self._set: # premiere fois : une simulation de plus
            self._set.add(idx)
            self.seen.append(idx)
        return self.table[idx]

    def F(self, idx): # objectifs et violation de contraintes
        r = self(idx)
        return np.array([r["CD_counts"], r["SER_db"]]), r["cv"]
```

Pour pouvoir répéter les expériences sur des dizaines de graines aléatoires, les 512 profils sont évalués une fois (`build_table`), puis chaque méthode lit cette table *à travers le compteur*. C'est strictement équivalent à appeler les modèles, qui sont déterministes, et c'est ce qui permet de mesurer précisément le nombre de simulations.

La violation de contraintes. Chaque contrainte s'écrit $g_i \leq 0$; la violation totale est $\mathrm{CV} = \sum_i \max(0, g_i)$. Un profil est faisable si $\mathrm{CV} = 0$. Sur la grille, 392 profils sur 512 sont faisables : les 120 autres ont $t = 6$ % ou $m = 0$ (portance insuffisante à 2°).

Exercices

Corrigés en annexe A. Chercher d'abord sans les regarder.

Exercice 1.1. Pourquoi ne pas simplement additionner C_D (en counts) et SER (en dBm) pour se ramener à un seul objectif ? Donner deux raisons.

Pistes d'amélioration.

— Ajouter le mode « portance imposée » ($C_L = 0{,}4$) comme troisième scénario : l'incidence devient une sortie de l'aéro, et le couplage avec la SER par l'attitude devient réel.

— Remplacer la table pré-calculée par un vrai appel aux modèles dès que la SER du groupe SER sera disponible : l'interface est la même, seul le coût change.

Chapitre 2

Dominance et front de Pareto

V1 Itération V1 — Chaîne complète conforme à la note 4–5 nov. 2026

2.1 Dominance

Avec deux objectifs à minimiser, deux profils ne sont pas toujours comparables : l'un peut traîner moins et être plus visible. On compare donc composante par composante.

> **Définition (dominance de Pareto).** Le profil a **domine** le profil b si $f_k(a) \leq f_k(b)$ pour tous les objectifs k, avec au moins une inégalité stricte. Un profil faisable qu'aucun autre profil faisable ne domine est **non dominé**, ou Pareto-optimal. Leur ensemble est le **front de Pareto**.

Le front est la réponse complète d'un problème bi-objectif : il contient tous les compromis rationnels, et le choix entre eux appartient au concepteur, pas à l'algorithme.

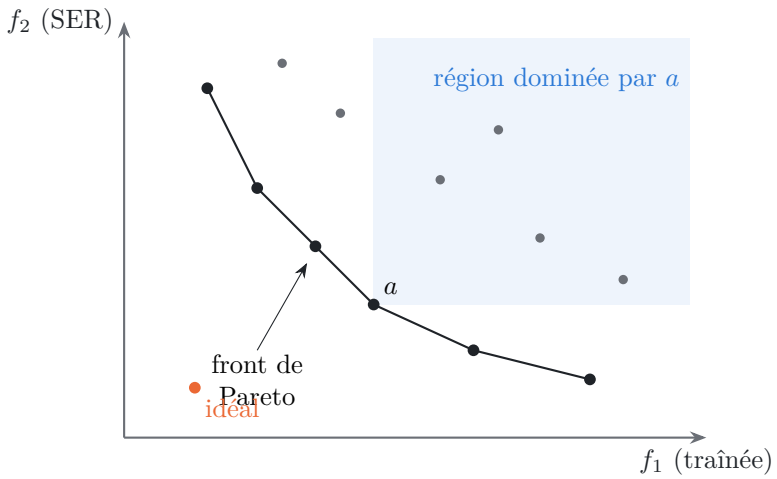


Figure 2.1 – Dominance. Les points noirs forment le front : aucun n'est battu sur les deux objectifs à la fois. Le point idéal (meilleure valeur de chaque objectif) n'est en général pas atteignable.

2.2 Ramener le problème à un seul objectif : scalarisation

Beaucoup de solveurs (et tout QUBO) ne savent minimiser qu'*une* fonction. Deux façons classiques de s'y ramener :

— la **somme pondérée** min $w\,\tilde{f}_1 + (1-w)\,\tilde{f}_2$, avec des objectifs normalisés $\tilde{f}_k \in [0,1]$, et un balayage de $w \in [0,1]$. Chaque w donne un point du front. Mais seuls les points sur l'*enveloppe convexe* du front, dits **supportés**, peuvent être obtenus ainsi : un point dans un creux du front n'est le minimum d'aucune somme pondérée ;
— l'**ε-contrainte** min f_1 sous $f_2 \leq \varepsilon$, avec un balayage de ε. Elle atteint tous les points du front, convexes ou non, mais ajoute une contrainte d'inégalité, donc des qubits d'écart dans un QUBO.

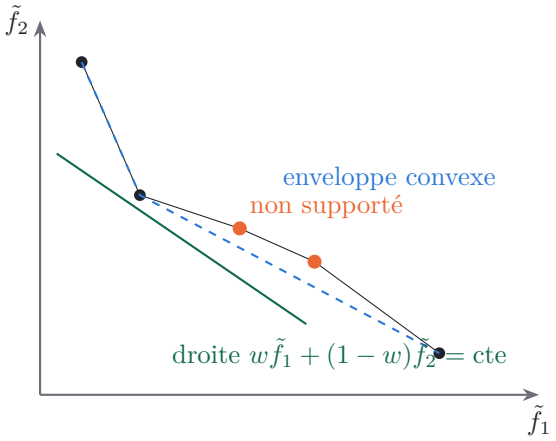


Figure 2.2 – La somme pondérée ne trouve que les points de l'enveloppe convexe du front. Les points orange, dans un creux, ne sont le minimum d'aucune combinaison $w\tilde{f}_1 + (1-w)\tilde{f}_2$.

2.3 Mesurer la qualité d'un front approché

Pour comparer des méthodes, il faut chiffrer la distance entre un front trouvé et le front exact.

— La **fraction du front exact trouvée** : proportion des profils Pareto-optimaux qui ont été simulés. C'est le critère le plus strict, et celui de la note d'intention (« atteindre le front exact »).
— L'**hypervolume** (HV) : l'aire de la région dominée par le front trouvé et bornée par un point de référence $\mathbf{r}$ (figure 2.3). Plus il est grand, mieux c'est ; il est maximal pour le front exact. On rapporte l'hypervolume trouvé à celui du front exact, sur des objectifs normalisés, avec $\mathbf{r} = (1{,}1;\ 1{,}1)$.
— L'**IGD** (distance générationnelle inversée) : la distance moyenne de chaque point du front exact au point trouvé le plus proche.

Dans le code : `optim/pareto.py`.

```python
def hypervolume_2d(F, ref):
    F = F[np.all(F < ref, axis=1)] # points qui dominent la reference
    F = F[pareto_front(F)]
    F = F[np.argsort(F[:, 0])] # tri selon f1, f2 decroissant
    hv, prev = 0.0, ref[1]
    for f1, f2 in F:
        hv += (ref[0] - f1) * (prev - f2) # un rectangle par point
        prev = f2
    return hv
```

Exercices

Corrigés en annexe A. Chercher d'abord sans les regarder.

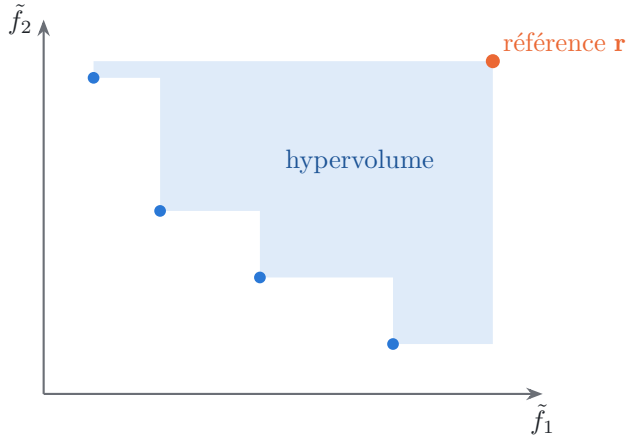


Figure 2.3 – L'hypervolume est l'aire dominée par le front et bornée par le point de référence. En 2D, c'est une somme de rectangles, après tri selon f_1.

Exercice 2.1. Trois profils ont pour objectifs $A = (60; -16{,}3)$, $B = (58; -16{,}2)$ et $C = (62; -16{,}1)$. Lesquels sont non dominés ? Calculer l'hypervolume de $\{A, B\}$ par rapport à $\mathbf{r} = (65; -16)$.

Exercice 2.2. Montrer qu'un point minimisant $w\tilde{f}_1 + (1-w)\tilde{f}_2$ pour un $w \in]0, 1[$ est forcément Pareto-optimal.

Pistes d'amélioration.

— Ajouter la scalarisation de Tchebycheff, $\min \max_k w_k |\tilde{f}_k - z^\star_k|$: elle atteint les points non supportés sans ajouter de contrainte, mais elle n'est pas quadratique (le max), donc pas directement un QUBO.

— Utiliser un troisième objectif (le moment de tangage, ou la SER à une autre fréquence) : le tri non dominé et l'hypervolume se généralisent, la visualisation devient plus difficile.

Chapitre 3

Le front exact par énumération

V1 Itération V1 — Chaîne complète conforme à la note 4–5 nov. 2026

3.1 Pourquoi on peut, et pourquoi c'est précieux

512 profils à environ 10 ms chacun (aéro + SER) : l'énumération exhaustive prend quelques secondes. Elle donne le front de Pareto *exact*, ce qui change la nature de la comparaison : on ne se demande plus quelle méthode trouve le meilleur front, mais combien de simulations chacune dépense pour retrouver un front connu. C'est exactement la question de la note d'intention, et elle n'a de sens que sur un cas réduit.

L'algorithme. On évalue tous les profils, on garde les faisables, on applique le tri non dominé et on conserve le premier front. Le tri de Deb compare toutes les paires : $O(MN^2)$ comparaisons pour N points et M objectifs, soit environ $1{,}5 \cdot 10^5$ comparaisons ici, négligeable.

3.2 Les deux fronts

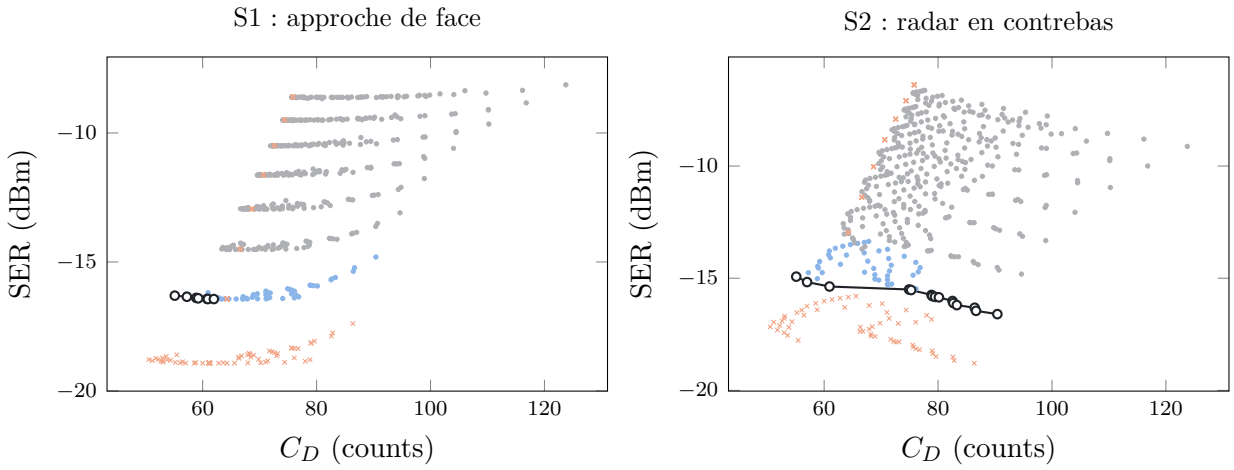


Figure 3.1 – Espace des objectifs pour les 512 profils. Points bleus : profils faisables à $t = 8$ % ; gris : autres épaisseurs faisables ; croix orange : infaisables. Ligne noire : front de Pareto exact. La SER est ordonnée par l'épaisseur, en bandes horizontales.

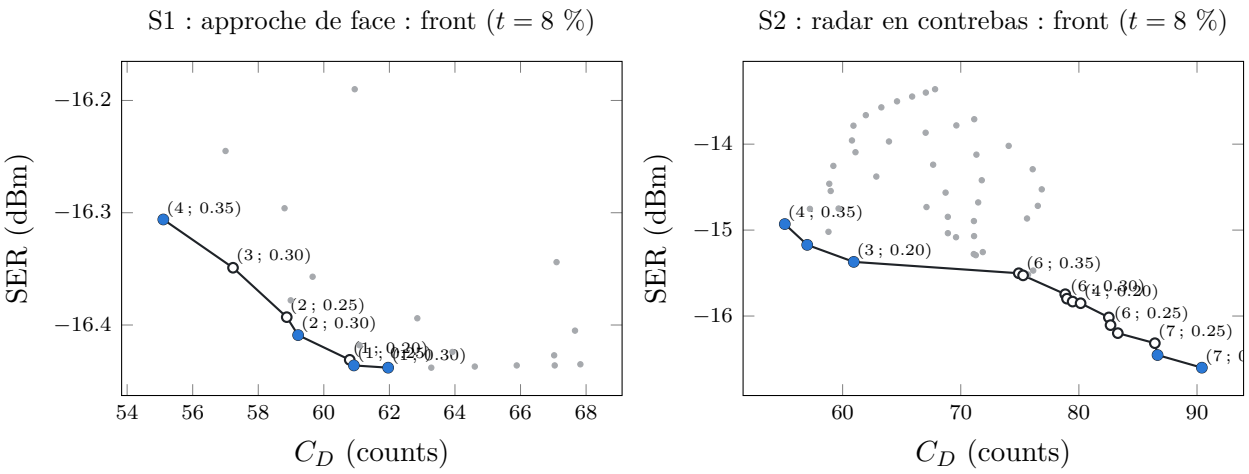


Figure 3.2 – Zoom sur les fronts exacts, tous à $t = 8$ %. Les étiquettes donnent (m %; p). Points bleus pleins : points supportés, atteignables par une somme pondérée (4 sur 7 dans S1, 5 sur 15 dans S2).

	S1 : approche de face	S2 : radar en contrebas
profils faisables	392	392
profils sur le front	7	15
dont supportés (somme pondérée)	4	5
épaisseur des profils du front	8 %	8 %
plage de C_D sur le front	55,1 à 62,0 counts	55,1 à 90,4 counts
plage de SER sur le front	0,13 dB	1,67 dB

3.3 Lecture physique

Les deux objectifs veulent la même épaisseur. Tous les profils du front ont $t = t_{\min} = 8$ %. La traînée croît avec t (survitesses, gradient adverse) et la SER aussi, par le rayon de bord d'attaque $r_{BA} \propto t^2$ (cours SER). Il n'y a donc aucun compromis sur l'épaisseur : la contrainte $t \geq 8$ % est active pour tous les profils optimaux.

Le compromis se joue sur la cambrure.

— Dans **S1**, le profil de traînée minimale est (4 %; 0,35) ; diminuer la cambrure réduit très légèrement la SER (0,13 dB au total) mais augmente la traînée. Le front est réel mais très court en SER : pour un concepteur, il se résume à « le profil de traînée minimale ».

— Dans **S2**, le radar voit l'intrados. Une cambrure plus forte, plus en avant, aplatit l'intrados sous le nez et *diminue* la SER de 1,7 dB, au prix de 35 counts de traînée. Le front est plus long, et c'est un vrai compromis.

Peu de points supportés. Seuls 4 points sur 7 (S1) et 5 sur 15 (S2) sont sur l'enveloppe convexe du front. Une méthode fondée sur la somme pondérée, comme notre QUBO, ne peut viser directement que ces points. Les autres devront être trouvés autrement (chapitre 6).

> **Dans notre projet.** C'est un résultat de la chaîne complète à porter devant l'équipe : avec la paramétrisation NACA à 4 chiffres et une SER d'approche de face, le compromis traînée/SER est *dégénéré en épaisseur*. Si le vrai solveur SER confirme la tendance, il faudra enrichir la paramétrisation (rayon de bord d'attaque indépendant, par exemple) ou le scénario radar pour

obtenir un problème plus intéressant.

Exercices

Corrigés en annexe A. Chercher d'abord sans les regarder.

Exercice 3.1. Dans S1, le profil (2 %; 0,25; 8 %) a $C_D = 58{,}87$ counts et SER $= -16{,}393$ dBm, et le profil (2 %; 0,30; 8 %) a $C_D = 59{,}22$ et SER $= -16{,}409$. Les deux sont sur le front. Pourquoi ? Un concepteur ferait-il vraiment la différence ?

Pistes d'amélioration.

— Ajouter une **tolérance** dans la dominance (ε-dominance) : deux profils dont les SER diffèrent de moins de 0,05 dB sont considérés comme égaux. Le front de S1 se réduirait à un ou deux profils, ce qui est la vraie réponse d'ingénierie.

— Vérifier la robustesse du front aux paramètres des modèles (critère de transition, x_{stop}, fréquence, secteur) : un point qui reste sur le front quels que soient ces paramètres est une conclusion solide.

Chapitre 4

NSGA-II : principe et implémentation

V1 Itération V1 — Chaîne complète conforme à la note 4–5 nov. 2026

4.1 Pourquoi un algorithme génétique

La note d'intention retient NSGA-II (Deb et al., 2002) pour trois raisons : il ne demande aucun gradient, il est robuste aux discontinuités (en SER, une facette qui passe de l'ombre à la lumière fait sauter l'objectif), et il produit un *ensemble* de solutions à chaque génération, donc une approximation de tout le front en une seule exécution. C'est la référence classique de l'optimisation multi-objectif, disponible dans pymoo.

4.2 Le principe

Un algorithme génétique fait évoluer une **population** de N solutions. À chaque génération :

1. on **sélectionne** des parents, en favorisant les meilleurs ;
2. on les **croise** et on les **mute** pour créer N enfants ;
3. on garde les N meilleurs parmi parents et enfants : c'est la **survie élitiste**.

Tout le problème, en multi-objectif, est de définir « meilleur » quand il n'y a pas d'ordre total. NSGA-II répond par deux critères, dans cet ordre : le **rang de non-dominance**, puis la **distance d'encombrement**.

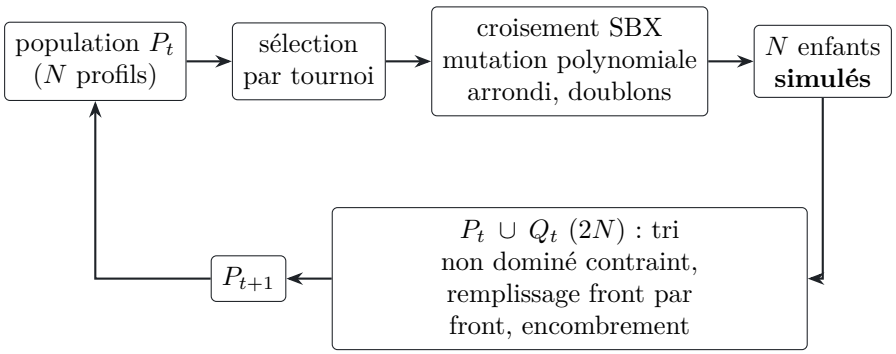


Figure 4.1 – Une génération de NSGA-II. Seuls les enfants coûtent des simulations ; les doublons sont éliminés avant d'être simulés.

4.3 Les deux critères de classement

Le tri non dominé. On répartit la population en fronts successifs $\mathcal{F}_1, \mathcal{F}_2, \ldots$: $\mathcal{F}_1$ contient les solutions non dominées, $\mathcal{F}_2$ celles qui ne sont dominées que par $\mathcal{F}_1$, etc. Le rang d'une solution est l'indice de son front. L'algorithme de Deb compte, pour chaque solution, combien d'autres la dominent, puis « épluche » les fronts : $O(MN^2)$ comparaisons.

La distance d'encombrement. À rang égal, on préfère les solutions isolées, pour bien répartir la population le long du front. Pour chaque objectif, on trie le front, et on ajoute à chaque point la distance normalisée entre ses deux voisins ; les extrémités reçoivent $+\infty$ pour être toujours conservées :

$$d_i = \sum_{k=1}^{M} \frac{f_k^{(i+1)} - f_k^{(i-1)}}{f_k^{\max} - f_k^{\min}}.$$

Les contraintes : règle de Deb. Pour comparer deux solutions : une faisable bat toujours une infaisable ; entre deux infaisables, la plus petite violation CV gagne ; entre deux faisables, on applique la dominance. Il n'y a ni pénalité ni coefficient à régler.

Dans le code : `optim/nsga2.py`, classement contraint.

```
def constrained_ranks(F, CV):
    feas = np.where(CV <= 0)[0]
    r = 0
    for front in non_dominated_sort(F[feas]): # faisables : fronts de Pareto
        ids = feas[front]
        rank[ids] = r
        crowd[ids] = crowding_distance(F[ids])
        r += 1
    infeas = np.where(CV > 0)[0]
    for i in infeas[np.argsort(CV[infeas])]: # infaisables : apres, par violation croissante
        rank[i] = r
        r += 1
    return rank, crowd
```

4.4 Les opérateurs, sur des variables entières

Nos variables sont des indices de 0 à 7. Comme pymoo, on les relâche en réels dans $[-0{,}5;\ 7{,}5]$, on applique des opérateurs continus, puis on arrondit.

Croisement SBX (*simulated binary crossover*). Pour chaque variable, avec probabilité $\frac{1}{2}$, les deux enfants sont placés symétriquement autour de la moyenne des parents, à une distance β fois leur écart, où β suit une loi qui se concentre près de 1 quand η_c est grand :

$$\beta = \begin{cases} (2u)^{1/(\eta_c+1)} & u \le \frac{1}{2} \\ \left(\frac{1}{2(1-u)}\right)^{1/(\eta_c+1)} & u > \frac{1}{2} \end{cases}, \qquad u \sim \mathcal{U}(0,1).$$

Mutation polynomiale. Avec probabilité $1/n_{var}$ par variable, un petit saut de loi en puissance, d'amplitude réglée par η_m.

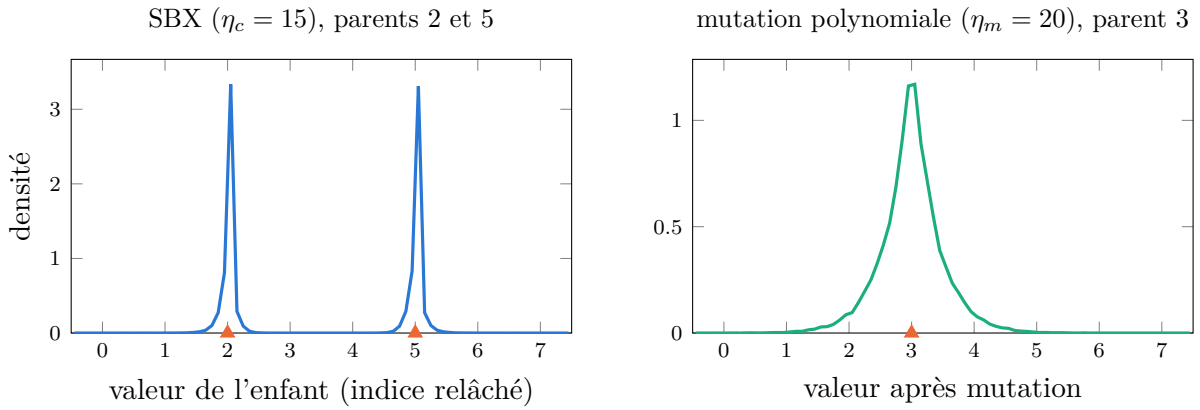


Figure 4.2 – Distribution des enfants pour les réglages usuels ($\eta_c = 15$, $\eta_m = 20$). Après arrondi, l'enfant SBX reprend la valeur d'un des parents dans 99,7 % des cas, et la mutation laisse l'indice inchangé dans 74 % des cas, sinon le décale de ± 1.

> **Piège.** Avec des variables à 8 niveaux, l'arrondi transforme ces opérateurs continus. SBX avec $\eta_c = 15$ devient un *échange de gènes* (chaque enfant hérite la valeur de l'un ou l'autre parent, variable par variable), et c'est la mutation qui explore, par pas de ± 1. Ce n'est pas un défaut, mais il faut le savoir : régler η_c ne change presque rien ici.

Doublons. Un enfant identique à un individu de la population, ou à un autre enfant, est rejeté avant d'être simulé. Sans cela, la population se remplirait de copies d'un même bon profil, et des simulations seraient gaspillées.

4.5 Vérification contre pymoo

Le script `optim/check_pymoo.py` lance l'implémentation de pymoo (installée dans `Code/.venv`) sur le même problème compté, avec les mêmes réglages ($N = 16$, SBX $\eta_c = 15$, mutation $\eta_m = 20$, arrondi, doublons éliminés). Sur 20 graines, le nombre médian de simulations pour retrouver tout le front est :

	notre NSGA-II	pymoo
S1 (7 profils)	106,5	95,0
S2 (15 profils)	106,0	110,5

Les deux implémentations se comportent de la même façon, aux fluctuations entre graines près.

Exercices

Corrigés en annexe A. Chercher d'abord sans les regarder.

Exercice 4.1. Calculer la distance d'encombrement des trois points d'un front (0; 1), (0,3; 0,5), (1; 0). Lequel serait éliminé en premier si l'on devait n'en garder que deux ? Pourquoi les extrémités reçoivent-elles $+\infty$?

Exercice 4.2. Avec $\eta_c = 15$ et des parents d'indices 2 et 5, quelle valeur de β faudrait-il pour qu'un enfant arrondi soit égal à 3 ? Avec quelle probabilité cela arrive-t-il ?

Pistes d'amélioration.

— Utiliser des opérateurs *entiers* natifs (croisement uniforme, mutation par décalage de ± 1 ou par tirage), plus lisibles que SBX arrondi.

— Utiliser un codage binaire (les 9 bits du QUBO) avec croisement à un point et mutation par bit : c'est la version « génétique » la plus proche de la formulation quantique.

— Passer à NSGA-III ou SMS-EMOA si l'on ajoute des objectifs.

Chapitre 5

NSGA-II sur notre problème

V1 Itération V1 — Chaîne complète conforme à la note 4–5 nov. 2026

5.1 Le protocole

Chaque expérience est répétée sur **20 graines aléatoires**, car NSGA-II est stochastique : un seul essai ne dit rien. On enregistre, après chaque génération, le nombre de simulations distinctes, la fraction du front exact déjà simulée et l'hypervolume relatif. On rapporte la médiane et les quartiles.

Le budget est mesuré en simulations, et seulement en simulations : une génération de NSGA-II coûte au plus N simulations, moins si certains enfants ont déjà été simulés à une génération antérieure.

5.2 Le réglage qui compte : la taille de population

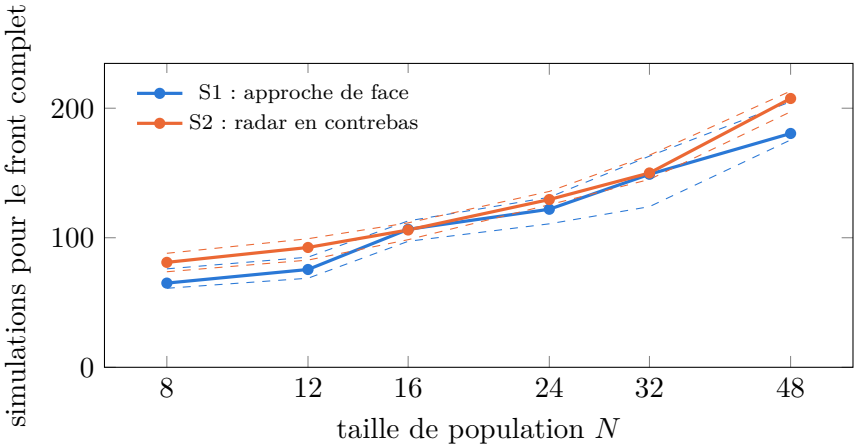


Figure 5.1 – Nombre de simulations nécessaires pour retrouver tout le front exact, selon la taille de population (médiane sur 20 graines, quartiles en tirets). Les petites populations sont nettement plus économes sur ce problème.

N	8	12	16	24	32	48
S1	65	76	107	122	149	181
S2	81	93	106	130	150	208

Sur un espace de 512 profils seulement, une grande population gaspille des simulations dans les premières générations, quand elle explore au hasard. Une petite population converge plus vite, et la mutation suffit à maintenir la diversité. La règle usuelle (N de 50 à 100) est pensée pour des espaces beaucoup plus grands. Dans la suite, on garde $N = 16$ comme réglage « par défaut », et on cite $N = 8$ comme NSGA-II « bien réglé ».

> **Piège.** Comparer une boucle QUBO soigneusement réglée à un NSGA-II avec ses réglages par défaut serait injuste. Une comparaison honnête doit régler les deux méthodes avec le même effort, ou au moins montrer la sensibilité de chacune à ses paramètres, comme ici.

5.3 Convergence

Les courbes de convergence de NSGA-II sont tracées au chapitre 7, avec celles des autres méthodes. Deux comportements ressortent :

— l'**hypervolume** monte vite : en quelques générations, NSGA-II a trouvé des profils proches du front, et l'hypervolume dépasse 99 % bien avant que le front soit complet ;
— la **fraction du front exact** progresse plus lentement, par paliers : les derniers profils du front sont des voisins très proches d'autres profils déjà trouvés (même t, cambrure voisine), et il faut une mutation précise pour tomber dessus.

Exercices

Corrigés en annexe A. Chercher d'abord sans les regarder.

Exercice 5.1. Avec $N = 48$ et 512 profils dans l'espace, quelle fraction de l'espace est simulée dès la population initiale ? Combien de générations faut-il au minimum pour dépasser 180 simulations ? Commenter le chiffre médian du tableau.

> **Pistes d'amélioration.**
> — Utiliser une population initiale construite (hypercube latin sur la grille) plutôt qu'aléatoire.
> — Arrêter NSGA-II avec un critère réaliste, qui n'utilise pas le front exact : stagnation de l'hypervolume pendant k générations. C'est ce qu'il faudrait faire sur un cas industriel, où le front exact n'est pas connu.

Chapitre 6

La boucle QUBO bi-objectif

V1 Itération V1 — Chaîne complète conforme à la note 4–5 nov. 2026

6.1 L'algorithme

La note d'intention décrit une boucle itérative : simuler quelques profils, ajuster un modèle quadratique, en tirer un QUBO, le résoudre par QAOA ou recuit, simuler les meilleurs candidats, recommencer. Le cours quantique l'a appliquée à la seule traînée. Ici, on l'étend aux deux objectifs et aux contraintes.

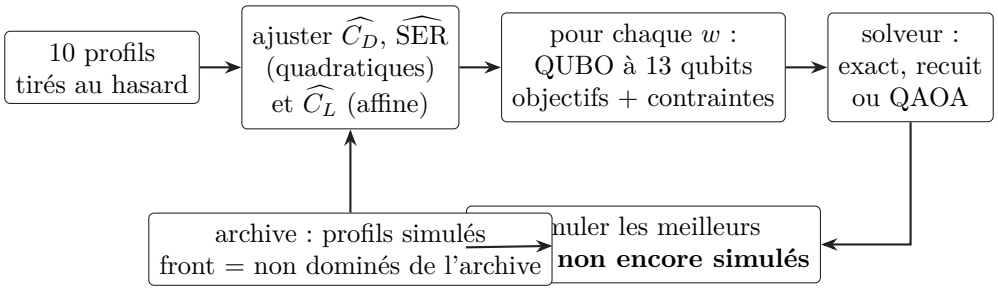


Figure 6.1 – La boucle bi-objectif. Le front rendu n'est formé que de profils réellement simulés, comme l'exige la note d'intention.

6.2 Étape 1 : les modèles de substitution

On ajuste par moindres carrés, sur les profils faisables déjà simulés, deux polynômes quadratiques en variables normalisées $z_i = 2i_i/7 - 1$ (10 coefficients chacun, « famille A » du cours aérodynamique) :

$$\widehat{C_D}(z),\ \widehat{\mathrm{SER}}(z) = c_0 + \sum_i c_i z_i + \sum_{i \le l} c_{il} z_i z_l.$$

Pour la contrainte de portance, on ajuste un modèle *affine* $\widehat{C_L}$ sur les profils valides : C_L est quasi affine en (m, α), et une contrainte doit rester de degré 1 pour que sa pénalité soit quadratique.

Une petite régularisation (ridge, 10^{-6}) évite les systèmes mal conditionnés quand peu de points sont disponibles. Avec 23 profils faisables, sur le scénario S2, le modèle de SER atteint $R^2 = 0{,}996$ sur toute la grille, celui de traînée $R^2 = 0{,}86$: la SER, lisse, est plus facile à modéliser que la traînée et ses bascules de transition.

6.3 Étape 2 : un QUBO par poids

L'objectif. Un QUBO n'a qu'une fonction de coût : on utilise une somme pondérée des deux modèles, normalisés par la plage des valeurs déjà observées,

$$f_w = w\,\frac{\widehat{C_D}}{\Delta C_D} + (1-w)\,\frac{\widehat{\mathrm{SER}}}{\Delta \mathrm{SER}}, \qquad w \in \{0;\ 0{,}25;\ 0{,}5;\ 0{,}75;\ 1\}.$$

Comme la substitution $z = a_0 + A\,\mathbf{b}$ est linéaire, on combine d'abord les coefficients, puis on développe en bits une seule fois (`surrogate_to_qubo`). On obtient 9 qubits.

La contrainte d'épaisseur : un qubit auxiliaire. $t \geq 8$ % interdit l'indice $i_t = 0$, c'est-à-dire les trois bits d'épaisseur nuls. La pénalité naturelle, $P(1-b_0)(1-b_1)(1-b_2)$, est *cubique* : ce n'est pas un QUBO (c'est le piège du degré vu dans le cours quantique). On la rend quadratique par la **réduction de Rosenberg**. On introduit un qubit y censé valoir x_1x_2, avec $x_1 = 1-b_1$ et $x_2 = 1-b_2$, et la pénalité

$$P\,(1-b_0)\,y + 2P\big[x_1x_2 - 2x_1y - 2x_2y + 3y\big].$$

Le crochet vaut 0 si $y = x_1x_2$ et au moins 1 sinon. Avec un poids $2P > P$, le minimum sur y est atteint pour $y = x_1x_2$, et on retrouve exactement la pénalité cubique. Le code vérifie l'égalité sur les 8 combinaisons par énumération.

La contrainte de portance : des qubits d'écart. $\widehat{C_L}(\mathbf{b}) \geq 0{,}3$ est une inégalité. On la transforme en égalité $\widehat{C_L}(\mathbf{b}) - 0{,}3 - s = 0$ avec une variable d'écart $s \geq 0$ codée sur 3 qubits, et on pénalise le carré :

$$P_{C_L}\big(\widehat{C_L}(\mathbf{b}) - 0{,}3 - s\big)^2, \qquad s = \frac{s_{\max}}{7}\sum_{j=0}^{2} 2^j s_j.$$

Comme $\widehat{C_L}$ est affine, le carré est bien quadratique. Contrepartie : s ne prend que 8 valeurs, et le résidu minimal n'est pas toujours nul. La pénalité ajoute donc un petit biais aux profils faisables, d'autant plus faible que le pas de s est fin.

Dans le code : `optim/qubo_loop.py`.

```
def build_qubo(coef_cd, coef_ser, coef_cl, w, scale_cd, scale_ser, cons, P=3.0, n_slack=3):
    coef = w * coef_cd / scale_cd + (1 - w) * coef_ser / scale_ser
    Q, c = surrogate_to_qubo(coef, BITS, LEVELS) # 9 qubits
    Q = grow(Q, 1 + n_slack) # + y + 3 bits d'ecart
    Q, c = penalty_index_nonzero(Q, c, bits=(6, 7, 8), y=9, P=P) # t >= 8 %
    const, lin = lin_bits(coef_cl) # C_L affine -> forme affine en bits
    Q, c = penalty_linear_geq(Q, c, lin, const, 0.3, [10, 11, 12], s_max, 25 * P) # C_L >= 0.3
    return Q, c, 13
```

6.4 Étape 3 : résoudre et choisir les candidats

Trois solveurs sont disponibles, de même interface : ils renvoient une liste de profils classés par énergie croissante.

— **exact** : énumération des $2^{13} = 8\,192$ états. Il sert de référence, en isolant l'effet du modèle de substitution de celui du solveur ;
— **recuit simulé** : 40 lectures, 150 balayages, comme le `SimulatedAnnealingSampler` d'Ocean ;

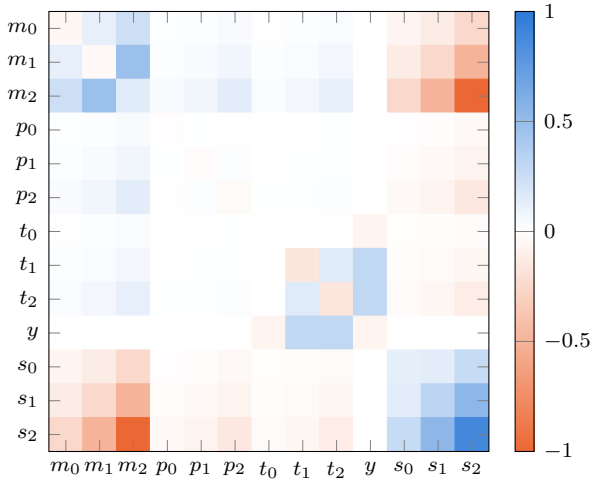


Figure 6.2 – Un QUBO de la boucle (scénario S2, $w = 0{,}5$, après 30 simulations), normalisé par son plus grand coefficient. Le bloc 9×9 en haut à gauche vient des modèles et du couplage avec la contrainte de portance ; la colonne y et le bloc s viennent des pénalités. Au total, 69 couplages non nuls, soit 138 CNOT par couche de QAOA.

— **QAOA** : profondeur $p = 2$, énergie normalisée, 3 départs pour COBYLA, puis 512 tirs ; on trie les chaînes distinctes obtenues par leur énergie recalculée classiquement.

Pour chaque poids, on retient le meilleur profil *qui n'a pas encore été simulé*. Ce détail est essentiel : sans lui, la boucle proposerait indéfiniment le même optimum du modèle, déjà connu.

Les points non supportés. La somme pondérée ne vise que les 4 ou 5 points supportés du front (chapitre 3). La boucle trouve pourtant les autres, pour deux raisons : l'exclusion des profils déjà simulés pousse vers les deuxièmes, troisièmes meilleurs, qui sont souvent des voisins du front ; et l'imperfection du modèle quadratique déplace les minima d'une itération à l'autre. C'est une exploration involontaire mais efficace.

6.5 Étape 4 : simuler, archiver, recommencer

Les candidats sont simulés (le compteur avance), ajoutés à l'archive, et le front courant est recalculé sur les seuls profils simulés. On s'arrête quand le budget est atteint, ou, pour la mesure de performance, dès que tout le front exact a été simulé.

Exercices

Corrigés en annexe A. Chercher d'abord sans les regarder.

Exercice 6.1. Vérifier la réduction de Rosenberg : pour $x_1 = x_2 = 1$, comparer la pénalité obtenue pour $y = 0$ et $y = 1$, et montrer que le minimum vaut $P(1 - b_0)$. Faire de même pour $x_1 = 1$, $x_2 = 0$.

Exercice 6.2. Combien de qubits faudrait-il si l'on codait la contrainte $i_t \geq 1$ par une variable d'écart, comme celle de portance, au lieu du qubit auxiliaire ? Quel avantage a la réduction de Rosenberg ici ?

Pistes d'amélioration.

— Remplacer la somme pondérée par une ε**-contrainte** $\min \widehat{C_D}$ sous $\widehat{\mathrm{SER}} \leq \varepsilon$: elle vise aussi les points non supportés, au prix de qubits d'écart supplémentaires.

— Pondérer les moindres carrés vers les meilleurs profils (modèle *local*, région de confiance), pour que le modèle soit précis là où se trouve le front.

— Choisir les candidats par un critère d'*amélioration attendue de l'hypervolume*, comme en optimisation bayésienne multi-objectif (ParEGO, EHVI) : c'est le standard actuel des méthodes à modèle de substitution, auquel la boucle QUBO devrait être comparée.

— Remplacer la variable d'écart de C_L, qui introduit un biais, par une réduction exacte quand la contrainte se ramène à une borne sur une variable (ici, $C_L \geq 0{,}3$ équivaut à $m \geq 1$ % sur la grille).

Chapitre 7

Comparaison de toute la chaîne

V1 Itération V1 — Chaîne complète conforme à la note 4–5 nov. 2026

7.1 Le protocole

Toutes les méthodes passent par le même compteur et sont arrêtées dès que le front exact est entièrement simulé (budget maximal : 150 simulations pour la boucle). On répète chaque expérience sur 20 graines (10 pour QAOA, plus coûteux à simuler) et on rapporte la médiane et les quartiles de deux grandeurs :

— le nombre de simulations pour avoir simulé **tout le front exact** ;
— le nombre de simulations pour atteindre **99 % de l'hypervolume exact**, c'est-à-dire un front pratiquement aussi bon que le vrai.

Les méthodes comparées : tirage aléatoire sans remise (la référence naïve), NSGA-II ($N = 16$, et $N = 8$ pour le réglage optimal du chapitre 5), et la boucle QUBO avec trois solveurs : exact, recuit simulé, QAOA $p = 2$. Le script est `optim/experiences_pipeline.py` ; il tourne environ une heure, dont l'essentiel pour QAOA.

7.2 Les courbes de convergence

7.3 Le tableau de synthèse

	S1 (7 profils)		S2 (15 profils)	
	front complet	HV 99 %	front complet	HV 99 %
tirage aléatoire	454	222	494	274
NSGA-II, $N = 16$	106,5	95	106	89
NSGA-II, $N = 8$ (réglé)	65	—	81	—
boucle QUBO, solveur exact	**35**	35	**67,5**	40
boucle QUBO, recuit simulé	40	30	85*	50
boucle QUBO, QAOA $p = 2$	**35**	30	80	**37,5**

Médianes en nombre de simulations distinctes. *Une graine sur 20 n'a pas complété le front dans le budget de 150 simulations.

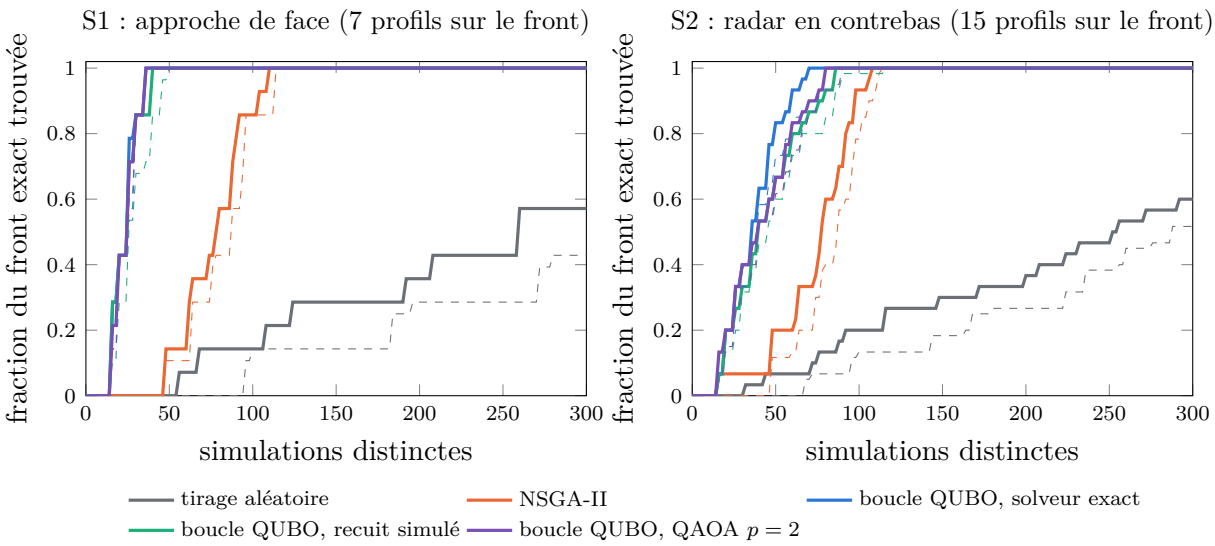


Figure 7.1 – Fraction du front exact déjà simulée, en fonction du nombre de simulations (médiane ; tirets : premier quartile).

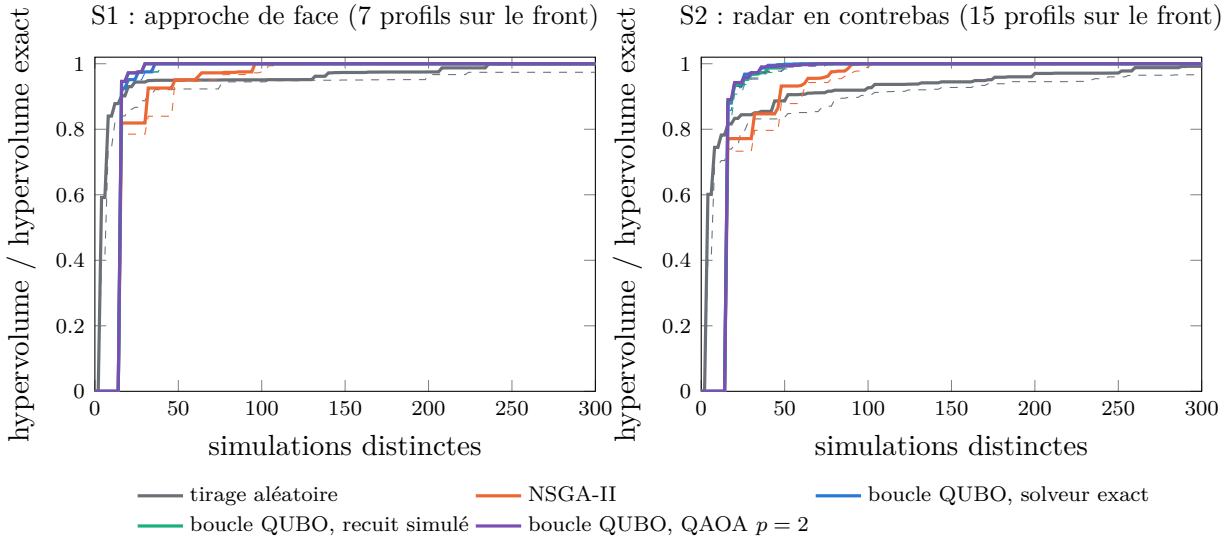


Figure 7.2 – Hypervolume relatif des profils simulés. La boucle atteint très vite un front presque optimal ; c'est la complétion des derniers points qui coûte.

7.4 Lecture

1. Toutes les méthodes battent le hasard, de loin. Pour avoir tout le front, le tirage aléatoire doit simuler presque toute la grille (454 et 494 profils sur 512) : les derniers profils du front sont trouvés au hasard parmi les tout derniers tirages.

2. La boucle QUBO économise des simulations, surtout sur le front court. Dans S1, elle retrouve le front en 35 simulations, trois fois moins que NSGA-II par défaut et presque deux fois moins que NSGA-II bien réglé. Dans S2, l'écart se réduit : 67,5 contre 106 et 81. Face à un NSGA-II réglé, le gain n'est plus que de 17 % avec le solveur exact, et devient nul avec QAOA.

3. Trouver un bon front est facile ; le compléter coûte. Pour atteindre 99 % de l'hypervolume, la boucle n'a besoin que de 30 à 50 simulations dans les deux scénarios, deux fois moins que NSGA-II. La complétion du front de S2 est plus coûteuse parce que 10 de ses 15 profils ne sont pas supportés : la somme pondérée ne les vise jamais directement, et la boucle ne les trouve qu'indirectement (exclusion des profils déjà simulés, erreurs du modèle quadratique).

4. Le solveur quantique ne change presque rien. Avec 13 qubits, le QUBO est résolu exactement en une fraction de seconde. QAOA ($p = 2$) et le recuit renvoient parfois un candidat un peu moins bon que l'optimum du QUBO, ce qui ajoute quelques itérations : +0 à +13 simulations pour QAOA, +5 à +18 pour le recuit. **Tout le gain de la boucle vient du modèle de substitution**, qui exploite la régularité des deux objectifs ; le solveur n'est qu'un composant interchangeable.

7.5 Ce que coûterait une vraie machine

Une itération de la boucle avec QAOA, c'est 5 QUBO (un par poids), chacun optimisé par COBYLA avec 3 départs, soit environ 600 évaluations de l'énergie moyenne. Sur un ordinateur quantique, chaque évaluation est une estimation statistique sur des centaines à des milliers de tirs. Avec environ 14 itérations pour S2 :

$$14 \times 5 \times 600 \approx 4 \cdot 10^4 \text{ évaluations} \quad \Rightarrow \quad \text{de l'ordre de } 10^7 \text{ exécutions de circuit à 1 000 tirs.}$$

Chaque circuit compte environ 69 couplages, donc 138 CNOT par couche et 276 pour $p = 2$, avant tout routage sur la topologie réelle. C'est faisable, mais coûteux et bruité, pour un problème qu'un ordinateur portable résout exactement en une milliseconde. Le seul argument quantique tenable porte donc sur le passage à l'échelle, et c'est ce que doit mesurer l'itération suivante.

À retenir.
- La boucle QUBO retrouve le front exact avec 1,2 à 3 fois moins de simulations que NSGA-II, et atteint 99 % de l'hypervolume deux fois plus vite.
- Ce gain vient du modèle de substitution ; le solveur (exact, recuit, QAOA) ne change presque rien à 13 qubits.
- La somme pondérée laisse de côté les points non supportés, qui font l'essentiel du coût de complétion quand le front est long.

Dans notre projet. C'est le résultat de l'itération V1. Il répond à la question de la note d'intention, mais il montre aussi ses limites : le problème est presque dégénéré et trop petit pour que le quantique joue un rôle. D'où la reformulation de l'itération V2 (voir la fiche projet).

Exercices

Corrigés en annexe A. Chercher d'abord sans les regarder.

Exercice 7.1. À partir du tableau, calculer le facteur de gain de la boucle (solveur exact) par rapport à NSGA-II réglé ($N = 8$), pour chaque scénario. Pourquoi le gain diminue-t-il quand le front s'allonge ?

Exercice 7.2. On veut estimer l'énergie moyenne d'un circuit QAOA à $\pm 0{,}01$ près, l'énergie normalisée ayant un écart-type de l'ordre de 1. Combien de tirs faut-il par évaluation ? En déduire le nombre total de tirs pour une exécution de la boucle sur S2.

Pistes d'amélioration.

— Mesurer la boucle avec un critère d'arrêt réaliste (stagnation de l'hypervolume) plutôt qu'avec le front exact.

— Réutiliser les angles optimaux de QAOA d'une itération à l'autre : le nombre d'évaluations diminuerait fortement.

— Ajouter l'optimisation bayésienne multi-objectif comme concurrent classique de même nature que la boucle.

Chapitre 8

Limites et pistes

V1 Itération V1 — Chaîne complète conforme à la note 4–5 nov. 2026

8.1 Ce que ces résultats ne disent pas

— **Le cas est petit.** 512 profils et 3 variables : un modèle quadratique global y est presque suffisant, ce qui avantage la boucle. Sur un cas industriel (dizaines de variables, front inconnu), le modèle quadratique sera moins fidèle et la boucle devra être locale.
— **Le front est presque dégénéré.** Tous les profils optimaux ont la même épaisseur ; le problème effectif se joue sur 64 profils (cambrure, position). C'est une conséquence physique des modèles (cours SER), pas un défaut des méthodes, mais cela rend la comparaison moins discriminante.
— **La SER est provisoire.** Le modèle d'optique physique est lisse ; un vrai solveur peut introduire des discontinuités qui dégraderont le modèle quadratique, et donc la boucle, plus que NSGA-II.
— **Le critère d'arrêt utilise le front exact.** C'est légitime pour mesurer, impossible en pratique : sur un cas réel, il faut un critère de stagnation, et le nombre de simulations « utiles » sera plus grand.
— **QAOA est simulé exactement.** Sans bruit, sans limite de profondeur, avec 512 tirs : c'est le meilleur cas possible pour QAOA.

8.2 Passage à l'échelle

	Cas du hackathon	Cas industriel
variables	3 variables à 8 niveaux, 9 bits	20 à 50 paramètres de forme, 60 à 300 bits
qubits du QUBO	13 (9 + 1 auxiliaire + 3 d'écart)	100 à 500
modèle quadratique	10 coefficients, $O(d^2)$	200 à 1 300 coefficients : autant de simulations au minimum
front exact	512 évaluations	inaccessible
simulateur	10 ms	minutes à heures (RANS, MoM 3D)

Le message de la note d'intention reste valable : seuls le simulateur et le nombre de variables changent. Mais le coût d'ajustement du modèle croît comme le carré du nombre de variables physiques, et devient vite le facteur limitant de la boucle, bien avant la taille du QUBO.

8.3 Pistes, par rapport utilité / effort

1. **Comparer à une optimisation bayésienne multi-objectif classique** (même modèle de substitution, candidats choisis par amélioration attendue de l'hypervolume, résolue par énumération ou par un algorithme génétique) : c'est le vrai concurrent de la boucle QUBO, et la seule façon d'isoler ce qu'apporte la résolution quantique du QUBO.
2. **ε-contrainte** dans le QUBO, pour viser les points non supportés.
3. **Modèle local** (moindres carrés pondérés autour du front courant).
4. **Réutiliser les angles de QAOA** d'une itération à l'autre, et ajouter du bruit pour estimer la dégradation sur machine réelle.
5. **NSGA-II à variables entières ou binaires natives**, et réglage systématique (taille de population, taux de mutation).
6. **Paramétrisation plus riche** (rayon de bord d'attaque indépendant) pour obtenir un vrai compromis traînée/SER.

Dans notre projet. Pour le jury, trois messages. La chaîne complète fonctionne, de la géométrie aux deux fronts de Pareto, et chaque méthode est mesurée par le même compteur. Sur ce cas réduit, la boucle quantique-classique retrouve le front exact avec moins de simulations que NSGA-II, mais l'essentiel du gain vient du modèle de substitution, pas du solveur quantique. Enfin, la physique des modèles (épaisseur dominante) est un résultat en soi, qui guide le choix de la paramétrisation pour la suite.

Chapitre 9

V2 : le problème reformulé

V2 Itération V2 — Problème reformulé 5–6 nov. 2026

V2 Évolution V2 : pourquoi ce chapitre. La V1 (chapitres 1 à 8) a laissé trois conclusions : le compromis traînée/SER était dégénéré (tout le front à $t = t_{\min}$), le gain de la boucle venait du modèle de substitution et non du solveur, et le modèle aérodynamique rapide déplaçait l'optimum. La V2 s'écarte de la note d'intention pour corriger ces trois points ; ce chapitre en présente le problème et les résultats.

9.1 Ce qui change

	V1	V2
géométrie	NACA 4 chiffres (m, p, t), 512 profils, 9 bits	NACA 4 chiffres modifiée (m, p, t, I), 1 024 profils, 10 bits
aérodynamique	modèle rapide, $\alpha = 2°$ imposé	XFOIL, $C_L = 0{,}4$ imposé (croisière)
contraintes	$t \geq 8$ %, $C_L \geq 0{,}3$, validité	$t \geq 8$ %, $C_{L,\max} \geq 1{,}2$ (manœuvre), convergence d'XFOIL
SER	optique physique	méthode des moments TM (référence), optique physique en contrôle
QUBO	13 qubits	14 qubits (10 + 1 auxiliaire + 3 d'écart)
comparaison	arrêt au front exact	arrêt au front exact *et* arrêt réaliste (stagnation de l'hypervolume)

Le code est dans `optim/problem_v2.py` (table XFOIL) et `optim/experiences_v2.py` (comparaison). La boucle QUBO (`qubo_loop.py`) a été généralisée à des variables de tailles différentes ; sa non-régression sur la V1 est vérifiée graine par graine.

9.2 Le front exact

Le front n'est plus confiné à une épaisseur. Dans S1, il compte 4 profils :

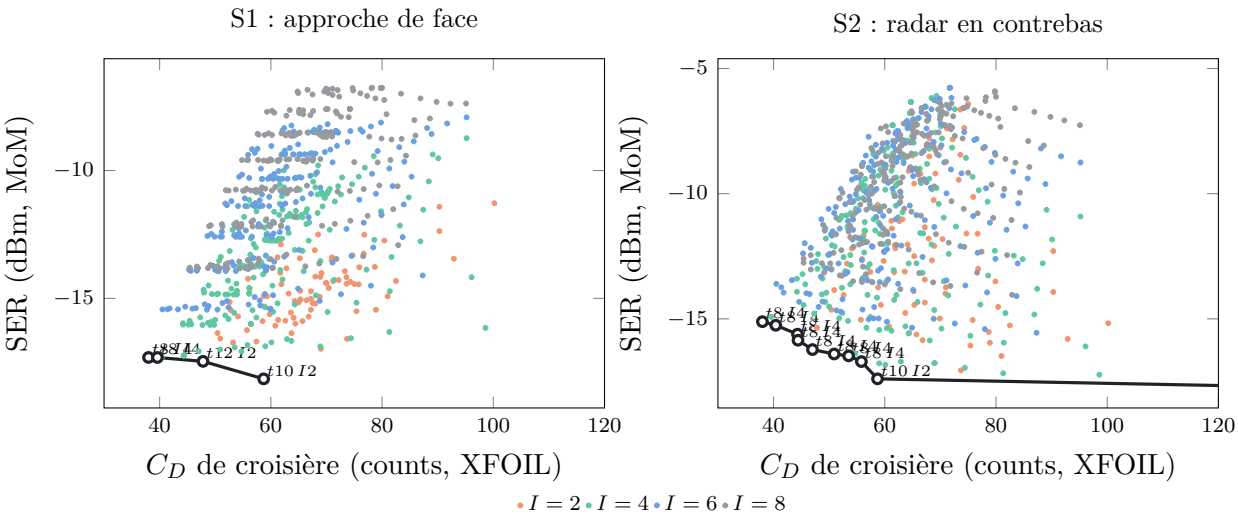


Figure 9.1 – Les 702 profils faisables de la V2, colorés par l'indice de rayon de bord d'attaque I, et le front exact (SER par méthode des moments). Les étiquettes donnent l'épaisseur et I des profils du front. Le point du front S2 à 319 counts (cambrure 7 % très en avant, décollement d'intrados) est hors du cadre.

m	p	t	I	C_D (counts)	SER (dBm)	$C_{L,\max}$
2 %	0,45	8 %	4	38,0	−17,31	1,30
2 %	0,55	8 %	4	39,6	−17,31	1,32
4 %	0,55	12 %	2	47,8	−17,46	1,20
5 %	0,45	10 %	2	58,7	−18,15	1,34

Le compromis a un sens physique clair. Le profil de traînée minimale est mince avec un nez modérément aigu ($I = 4$). Pour gagner en SER, il faut passer au nez le plus aigu ($I = 2$) ; mais un nez aussi aigu ne tient l'exigence de manœuvre qu'avec plus d'épaisseur et de cambrure, ce qui coûte de la traînée. Dans S2 (radar en contrebas), le front compte 10 profils et 3,4 dB d'écart de SER, contre 0,8 dB dans S1.

9.3 La comparaison des méthodes

Même protocole qu'au chapitre 7, avec 20 graines (5 pour QAOA, plafonné à 150 simulations) et deux critères d'arrêt.

	S1 (4 profils)		S2 (10 profils)	
	front complet	réussite	front complet	réussite
tirage aléatoire	911	100 %	972	100 %
NSGA-II, $N = 8$	—	0 %	—	0 %
NSGA-II, $N = 16$	—	5 %	—	5 %
boucle, solveur exact	**75**	100 %	**142**	100 %
boucle, recuit simulé	97,5	100 %	235	55 %
boucle, QAOA $p = 2$	105	100 %	—	60 %*

Médianes en simulations distinctes ; « — » : médiane non atteinte dans le budget (250, ou *150 pour QAOA).

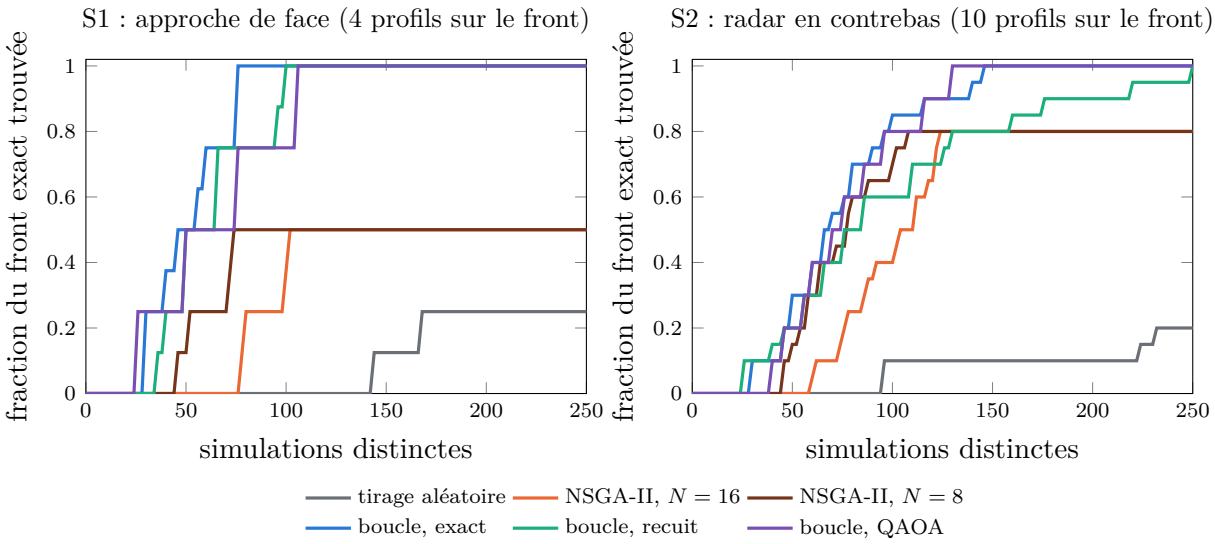


Figure 9.2 – V2 : fraction du front exact trouvée selon le nombre de simulations (médiane). NSGA-II progresse lentement et ne complète presque jamais le front dans le budget de 250 simulations.

Avec un critère d'arrêt réaliste. Sur un vrai problème, on ne connaît pas le front exact : on s'arrête quand l'hypervolume ne progresse plus (5 générations pour NSGA-II, 3 itérations pour la boucle). Médianes au moment de l'arrêt :

	S1			S2		
	simulations	front	HV	simulations	front	HV
NSGA-II, $N = 8$	78	50 %	0,937	91	65 %	0,975
NSGA-II, $N = 16$	137	50 %	0,937	151	80 %	0,978
boucle, exact	52	50 %	0,943	68	55 %	0,962
boucle, recuit	55	62 %	0,949	82	45 %	0,989
boucle, QAOA	75	50 %	0,937	105	80 %	1,000

9.4 Lecture

1. Sur un problème plus réaliste, la boucle domine nettement. NSGA-II, qui retrouvait le front V1 en 65 à 106 simulations, ne le complète presque plus dans un budget de 250 : 30 % des profils sont infaisables, et le front est coincé contre la contrainte de manœuvre, dans une petite région de l'espace. La boucle, elle, apprend la structure des deux objectifs et de la contrainte, et retrouve le front en 75 à 142 simulations.

2. Le gain reste celui du modèle, pas du solveur. Le solveur exact du QUBO reste le meilleur ; le recuit et QAOA, qui ne renvoient pas toujours l'optimum du QUBO, demandent plus d'itérations et échouent parfois dans le budget. À 14 qubits, le solveur quantique n'apporte rien que l'énumération ne fasse mieux.

3. L'arrêt réaliste rapproche les méthodes. Sans connaître le front exact, toutes s'arrêtent avec environ la moitié du front et 94 à 100 % de l'hypervolume : on obtient un front presque aussi bon que le vrai, mais pas tous ses profils. La boucle s'arrête plus tôt ; QAOA s'arrête plus tard, avec un front plus complet dans S2. Ce critère est celui qu'il faudrait présenter pour un cas industriel.

À retenir.
— Le problème V2 contient un vrai compromis traînée/SER, porté par l'épaisseur, la cambrure et le rayon de bord d'attaque.
— La boucle quantique-classique y retrouve le front exact quand NSGA-II n'y parvient presque jamais dans 250 simulations.
— Le solveur exact du QUBO reste le meilleur : le gain vient du modèle de substitution.

Exercices

Corrigés en annexe A. Chercher d'abord sans les regarder.

Exercice 9.1. Pourquoi le profil (4 %; 0,55; 12 %; $I = 2$) est-il sur le front de S1 alors que sa portance maximale (1,20) est juste à la limite de l'exigence ? Que deviendrait le front si l'on exigeait $C_{L,\max} \geq 1{,}3$?

Pistes d'amélioration.
— Comparer la boucle à une optimisation bayésienne multi-objectif classique : c'est le vrai concurrent de même nature, et la seule façon d'isoler ce que la formulation QUBO apporte.
— Ajouter l'ε-contrainte pour viser directement les points non supportés.
— Étudier la sensibilité du front à l'exigence de manœuvre (1,1 à 1,4) : c'est le paramètre qui crée le compromis.

Annexe A

Corrigés des exercices

V1 Itération V1 — Chaîne complète conforme à la note 4–5 nov. 2026

Chapitre 1

Ex. 1.1. (1) Les unités et les échelles sont arbitraires : un count de traînée et un décibel de SER n'ont aucune équivalence naturelle, et le résultat dépendrait entièrement de ce choix. (2) Une somme donne *un* profil, alors que la question posée est l'ensemble des compromis : le choix entre eux appartient au concepteur. On pourrait ajouter que la SER en dB est une échelle logarithmique, que l'on mélangerait avec une échelle linéaire.

Chapitre 2

Ex. 2.1. B a moins de traînée que A mais plus de SER : ni l'un ni l'autre ne domine. A domine C ($60 < 62$ et $-16{,}3 < -16{,}1$). Non dominés : A et B. Hypervolume, après tri selon f_1 : B donne $(65-58)\times(-16-(-16{,}2)) = 1{,}4$, puis A donne $(65-60)\times(-16{,}2-(-16{,}3)) = 0{,}5$, soit 1,9 (counts·dB).

Ex. 2.2. Soit $x^\star$ un minimum de $g = w\tilde{f}_1 + (1-w)\tilde{f}_2$ avec $0 < w < 1$. Si un point y dominait $x^\star$, on aurait $\tilde{f}_k(y) \le \tilde{f}_k(x^\star)$ pour les deux objectifs, dont une inégalité stricte, multipliée par un poids strictement positif. Donc $g(y) < g(x^\star)$, ce qui contredit la minimalité.

Chapitre 3

Ex. 3.1. Le premier a moins de traînée (58,87 contre 59,22), le second moins de SER ($-16{,}409$ contre $-16{,}393$) : aucun ne domine l'autre. Mais l'écart de SER, 0,016 dB, est très en dessous de toute précision de modèle ou de mesure. Un concepteur garderait le premier : c'est l'argument de l'ε-dominance.

Chapitre 4

Ex. 4.1. Selon f_1, le point du milieu a pour voisins 0 et 1 : contribution $(1-0)/1 = 1$. Selon f_2, de même : 1. Sa distance vaut 2 ; les extrémités valent $+\infty$. C'est donc le point du milieu qui serait

éliminé. Les extrémités reçoivent $+\infty$ pour que la population conserve toute l'étendue du front : sans elles, l'algorithme pourrait perdre les meilleurs profils pour chaque objectif pris isolément.

Ex. 4.2. La moyenne vaut 3,5 et la demi-distance 1,5 : l'enfant $3{,}5 - 1{,}5\beta$ s'arrondit à 3 si $\beta \le \frac{2}{3}$. Pour $u \le \frac{1}{2}$, $\beta = (2u)^{1/16} \le \frac{2}{3}$ exige $u \le \frac{1}{2}(\frac{2}{3})^{16} \approx 7{,}6 \cdot 10^{-4}$. Avec la probabilité $\frac{1}{2}$ de croiser cette variable, cela arrive environ 4 fois sur 10 000.

Chapitre 5

Ex. 5.1. $48/512 \approx 9{,}4$ % de l'espace dès l'initialisation. Après g générations, au plus $48(g+1)$ simulations : il faut $g \ge 3$ pour dépasser 180. La médiane observée, 180,5, correspond donc à environ quatre « paquets » de 48 : avec une grande population, on paie par paquets, même quand le dernier profil manquant est trouvé au début du paquet.

Chapitre 6

Ex. 6.1. $x_1 = x_2 = 1$: pour $y = 0$, le crochet vaut $x_1 x_2 = 1$, pénalité $2P$; pour $y = 1$, il vaut $1 - 2 - 2 + 3 = 0$, pénalité $P(1 - b_0)$. Le minimum est $P(1 - b_0)$, puisque $2P > P(1 - b_0)$. $x_1 = 1$, $x_2 = 0$: pour $y = 0$, tout est nul ; pour $y = 1$, le crochet vaut $-2 + 3 = 1$, pénalité $2P + P(1 - b_0)$. Le minimum est 0, comme $P(1 - b_0)x_1 x_2$.

Ex. 6.2. Il faudrait écrire $i_t - 1 - s = 0$ avec $s \in \{0, \dots, 6\}$, codé sur 3 bits : 3 qubits au lieu d'un, et davantage de couplages. La réduction de Rosenberg est exacte et ne coûte qu'un qubit, car la contrainte n'exclut qu'une seule valeur.

Chapitre 7

Ex. 7.1. S1 : $65/35 \approx 1{,}9$. S2 : $81/67{,}5 = 1{,}2$. Le gain diminue parce que la boucle, fondée sur une somme pondérée, ne vise directement que les points supportés (4 sur 7 dans S1, 5 sur 15 dans S2) : plus le front contient de points non supportés, plus la boucle dépend de l'exploration indirecte, moins efficace. NSGA-II, lui, entretient une population répartie le long du front et ne fait pas de différence entre points supportés et non supportés.

Ex. 7.2. L'erreur d'une moyenne empirique vaut $\sigma/\sqrt{N}$: $1/\sqrt{N} = 0{,}01$ donne $N = 10^4$ tirs. Avec environ $4 \cdot 10^4$ évaluations par exécution de la boucle, cela fait $4 \cdot 10^8$ tirs. Même à quelques milliers de tirs par seconde, il faut des jours de machine pour un problème résolu classiquement en une milliseconde : la boucle variationnelle elle-même est le goulot d'étranglement.

Chapitre 9 V2

Ex. 9.1. Il est sur le front parce qu'aucun profil faisable n'a à la fois moins de traînée et moins de SER : son nez très aigu ($I = 2$) lui donne une SER plus faible que tous les profils plus minces, et l'épaisseur de 12 % lui permet tout juste de tenir l'exigence de manœuvre. Avec $C_{L,\max} \ge 1{,}3$, il deviendrait infaisable ; le front perdrait ce compromis intermédiaire, et les profils à $I = 2$ restants devraient être encore plus épais ou plus cambrés, donc plus traînants. L'exigence de manœuvre pilote directement la forme du front : c'est pourquoi sa sensibilité est une piste prioritaire.